

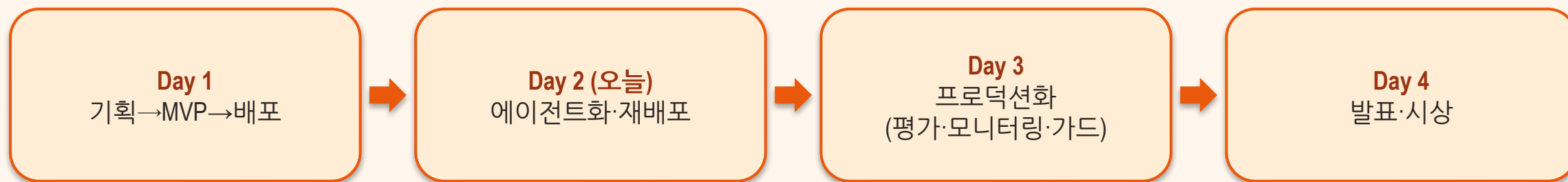
14차시 · 팀 프로젝트 (매일 저녁 19:00~21:00)

팀 프로젝트 Day 2 — 에이전트화 · 재배포

어제의 MVP를 오늘 밤 에이전트가 일하는 서비스로 — 같은 URL, 새로운 뇌

오늘은 몸통에 뇌를 얹는 밤

▶ 어제 URL이 없으면 오늘 붙일 몸통이 없다



오늘의 리듬 — 원칙 3

① 같은 레포·같은 URL

새 프로젝트 만들지 말 것 — 어제 URL을 유지

② main push = 자동 재배포

커밋·머지가 곧 배포 — 로컬만 고치고 끝내지 않기

③ Less is More

도구·에이전트 1개만 제대로

낮 차시 지도 — 재료 창고

▶ 오늘 밤 붙일 건 이 중 하나만 — 잠깐 본 걸 여기서 처음 붙인다

7·8차시

에이전트 루프 · 설계 원칙

9차시

도구 호출 · MCP

10·11차시

메모리 · RAG

12·13차시

멀티 · 계획 · 반성

SECTION 01

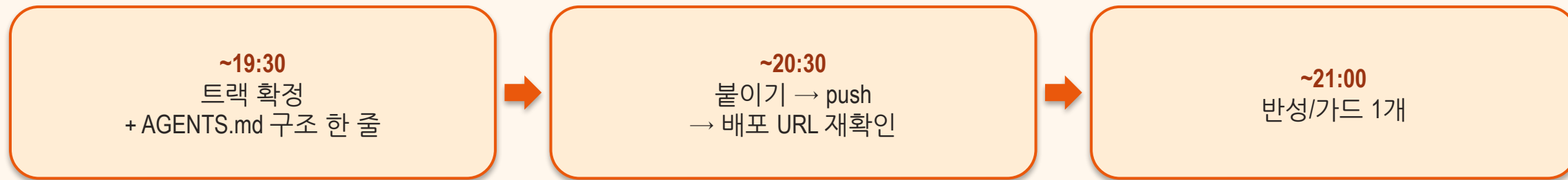
오늘 밤의 미션

목표 & 필수 미션 (DoD) — 4개 전부 필수

▶ 키·URL·구조 세 개 중 하나라도 빠지면 미완

- □ 에이전트 기능 1개가 배포된 서비스에서 동작
- □ AGENTS.md에 에이전트 구조 한 줄 — 입력 → 계획 → 도구/응답 → (반성) → 출력
- □ 같은 GitHub 레포 push → 같은 배포 URL 자동 갱신 — 전원 재접속 확인
- □ API 키는 서버/환경변수로만 — 프론트·커밋에 노출 금지

예시 타임라인



SECTION 02

STEP 1 · ~19:30 — 에이전트 트랙 확정

4가지 트랙

T1. 도구 호출 (9차시)

외부 데이터·행동이 필요할 때
— 검색·계산·API

T2. RAG (10·11차시)

우리만의 지식이 있을 때 —
문서·DB 검색

T3. Planner-Executor (12차시)

다단계 작업·품질이 중요할 때

T4. Reflection 루프 (13차시)

오답 위험·정답이 애매할 때

트랙 결정 질문 — 4개

▶ 4번을 답 못하면 트랙을 바꿔야 한다

1

어제 MVP에서 가장 아쉬웠던 부분은 무엇인가?

기능·정확도·속도 중 어디

2

그 아쉬움이 어느 트랙에 해당하나?

외부 데이터=T1 / 우리 문서=T2 / 다단계=T3 / 오답위험=T4

3

1시간 안에 붙일 수 있는 크기인가?

클수록 T1 → T2 → T3 → T4 순으로 어렵다

4

데모에서 '보일' 순간은 무엇인가?

답 못하면 '붙었는데 안 보인다'

AGENTS.md — 에이전트 구조 한 줄

Agent Loop

입력(사용자 발화) → [계획: LLM] → [도구/검색: T1~T4 중 하나] → [반성(있으면)] → 응답

- 트랙: T1 도구호출 (예: 웹검색 API)
- 실패 시 폴백: 검색 0건이면 원래 답으로
- 데모 순간: '∞를 물으면 실시간 결과가 카드로 뜬다'

SECTION 03

STEP 2 · ~20:30 — 붙이고 재배포

역할 분담 예시

▶ 환경변수·시크릿까지 손 대는 유일한 사람 = 배포·통합

에이전트 페어 (2명)

트랙 T1~T4 붙이기 — 드라이버·내비게이터 교대

UI 페어 (2명)

에이전트가 보이도록 화면 조정 — 로딩·에러 상태 포함

배포·통합 (1명)

환경변수 · 재배포 · URL 확인

코드 붙이기 — 서버 함수 최소 예시

▶ 키는 절대 브라우저에 노출 금지 — 프론트는 fetch로 프록시

```
// api/agent.js (Vercel Functions / Node)
import OpenAI from "openai";
const client = new OpenAI({ apiKey: process.env.OPENAI_API_KEY });
export default async function handler(req, res) {
  const { user_msg } = req.body;
  // T1: 도구 스키마 / T2: 문서 검색 / T3: 계획 / T4: 반성 — 택 1
  const r = await client.chat.completions.create({
    model: "gpt-4o-mini", messages: [/* ... */]
  });
  res.json({ reply: r.choices[0].message.content });
}
```

API 키 — 브라우저에 절대 노출 금지 3계명

▶ 키 유출 = 즉시 요금 사고

① 서버(Functions)에서만 호출

프론트는 `fetch("/api/agent")`로 프록시 — 키가 코드에 들어가지 않게

② Vercel Env / Actions Secrets에 저장

`.env`는 gitignore · Vercel 대시보드에 등록

③ 커밋 이력 검색

`git log -S sk-` — 실수로 넣었다면 키 폐기·재발급 후 커밋 정리

Vercel Env 등록 — 실전 3단계

▶ API 키를 코드에 안 넣고 배포

① 등록

대시보드: Project → Settings → Environment Variables → Add
(Name: `OPENAI\_API\_KEY` · Value: `sk-...` · Environments: **Production + Preview + Development** 다 체크)

② 로컬 개발

`env.local`에 같은 값 — **.gitignore**에 **`.env\*`** 필수 · 팀원 동기화는 `vercel env pull .env.local`

③ 반영

등록만으로 반영 안 됨 — **재배포 트리거**: 빈 커밋 push 또는 Deployments 탭에서 Redeploy

Vercel Env 등록 — 실전 3단계

▶ API 키를 코드에 안 넣고 배포

```
# .gitignore - 최소
.env
.env.local
.env*.local

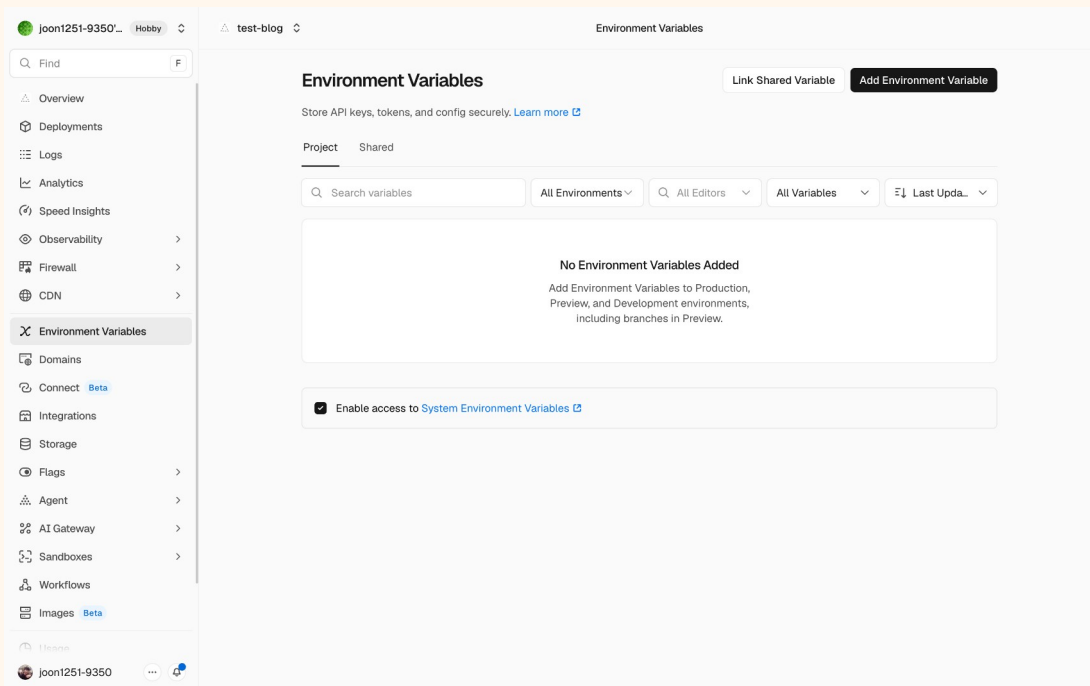
# 로컬 (한 번만)
echo "OPENAI_API_KEY=sk-..." > .env.local

# 팀원 동기화 (Vercel CLI)
npx vercel link          # 팀 프로젝트에 연결
npx vercel env pull .env.local # 등록된 값 내려받기

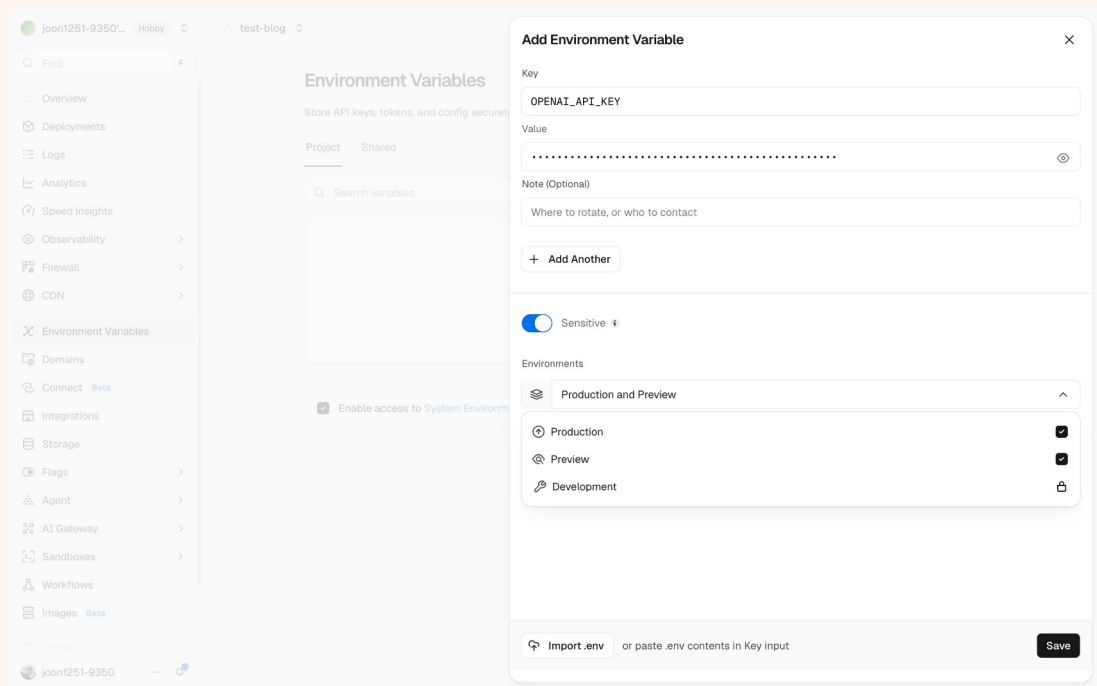
# 반영 (등록 후 반드시)
git commit --allow-empty -m "trigger redeploy" && git push
```

~20:30

Vercel Env 화면 — 등록 (① 페이지 · ② Add 폼)



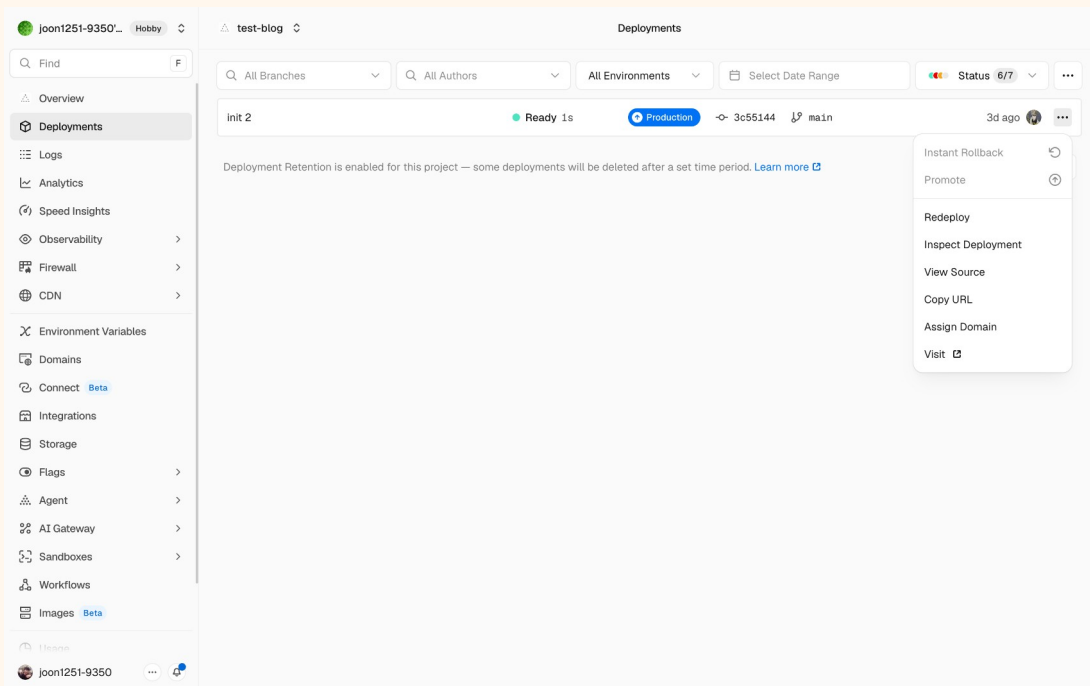
① Environment Variables → Add Environment Variable



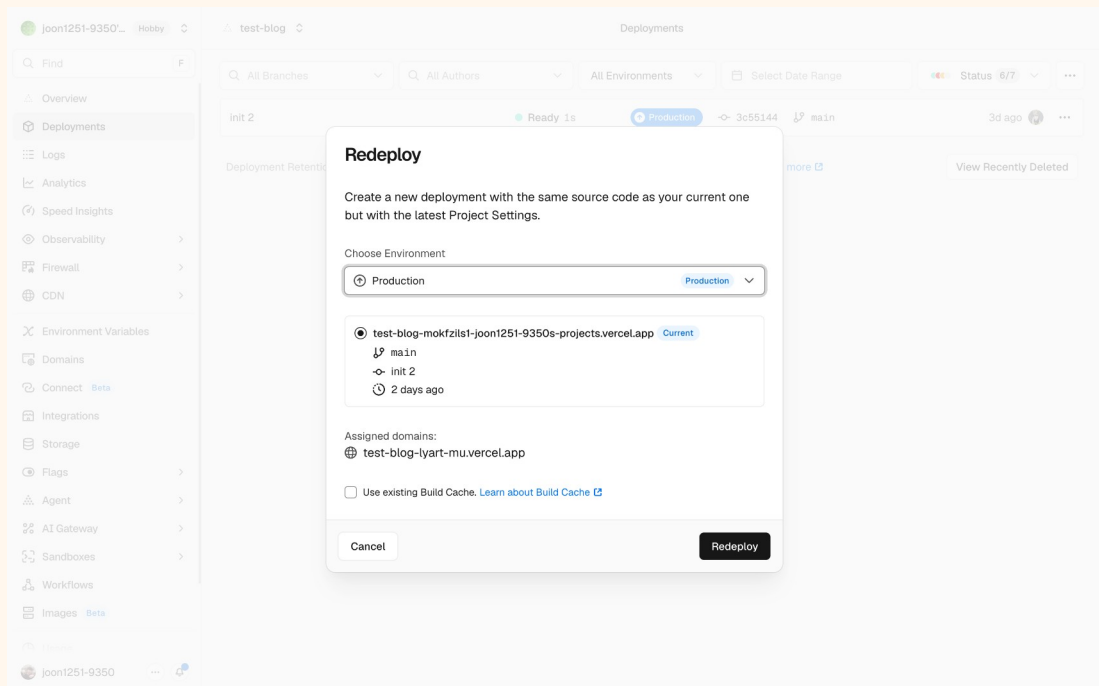
② Key/Value + Environments 3개 다 체크 (Sensitive ON이면 Development 잠금)

~20:30

Vercel Env 화면 — 재배포 (③ 메뉴 · ④ 확인)



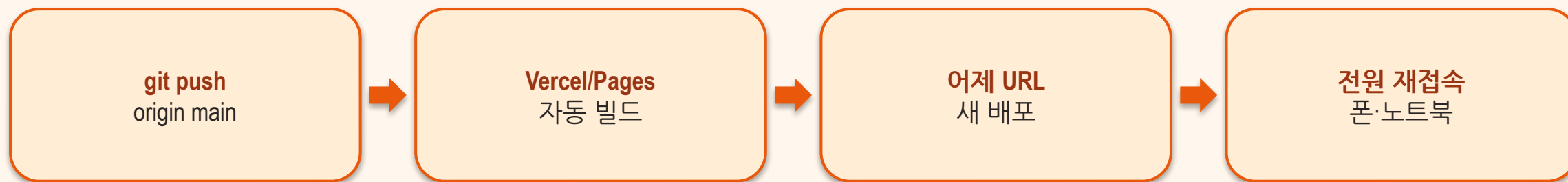
③ Deployments → 우측 ... → Redeploy



④ Production 선택 → Redeploy (Env 즉시 반영)

재배포 — 같은 URL 유지

▶ 새 Vercel 프로젝트 만들지 말 것 — 어제 프로젝트에 push만



실패 시 롤백: **Vercel Deployments** 탭 → 이전 배포 **Promote** — 어제 URL로 즉시 복귀.

실전 예시 · 어제 앱에 에이전트 없기 — 바뀌는 건 3곳뿐

▶ 프론트 코드엔 sk- 가 하나도 없다 — 개발자도구로 열어도 안 보인다

① 프론트 UI 조각 1개

버튼 + 출력 영역 `<section class="agent">` — 에이전트가 보이도록

② fetch 한 번

`fetch("/api/agent", { method:"POST", body: JSON })` — 프론트는 여기까지

③ 서버 함수 새 파일

`api/agent.js` 가 `process.env.OPENAI_API_KEY` 로 호출 — 키는 서버 안에서만

두 번째 키(Supabase·Firebase 등)도 코드에 안 넣기 · /api/config

▶ 정적 페이지는 브라우저가 env를 못 읽는다 → 서버 함수가 대신 내려준다

```
// api/config.js - 공개 URL·키를 env에서 내려줌 (서버 함수)
export default function handler(req, res) {
  res.json({ url: process.env.SUPABASE_URL,
             anonKey: process.env.SUPABASE_ANON_KEY });
}

// 프론트: 하드코딩 대신 받아서 생성
const { url, anonKey } = await (await fetch("/api/config")).json();
const db = createClient(url, anonKey);
```

공개돼도 되는 키(Supabase anon)라도 코드·커밋엔 안 넣는다 — 진짜 보안은 Supabase RLS · 진짜 비밀은 OpenAI 키뿐(서버 밖 금지).

~20:30

이번 앱의 환경변수 3개 — Vercel에 등록

▶ 비밀 키 1개 + 공개 키 2개 · 셋 다 코드엔 없고 Vercel Env에만

OPENAI_API_KEY

🔒 비밀 — 서버 함수 `/api/agent` 안에서만 사용, 프론트로 절대 안 나감

SUPABASE_URL

🌐 공개 — `/api/config` 가 프론트로 내려줌 (그래도 코드엔 안 넣음)

SUPABASE_ANON_KEY

🌐 공개(RLS로 보호) — `/api/config` 가 내려줌 · 실제 보안은 Supabase RLS

셋 다 **Settings** → **Environment Variables** → **Add** 로 등록 · **Production**·**Preview**·**Development** 3개 체크 · 등록 후 반드시 **재배포**

배포했는데 안 될 때 — 개발자도구로 3분 진단

▶ 등록만으론 반영 안 됨 — 재배포해야 새 프로세스가 env를 읽는다

1

Network 탭

우리 서버 `/api/agent` 요청만 보이고 `api.openai.com` 은 프론트에서 안 나간다 → 키 안전

2

`/api/config` 열어보기

응답이 `{}` 면 Env 미등록 — Vercel에 등록하고 재배포

3

Console 확인

`'supabaseUrl is required' · 'undefined'` → 같은 원인(Env). 등록·재배포 후 새로고침

벡터 DB 붙이기 — Supabase pgvector (SQL 한 번)

▶ 함정 ①: RLS 정책을 통과해도 GRANT 권한이 없으면 permission denied

```
-- Supabase SQL Editor 에서 한 번 실행
create extension if not exists vector;           -- pgvector 켜기
alter table memos add column embedding vector(1536); -- 임베딩 컬럼

create function match_memos(query_embedding vector(1536), match_count int)
  returns table(text text, similarity float) ...    -- 유사도 검색 함수

grant insert on memos to anon;  -- 정책(RLS)만으론 부족 · 권한도 따로!
```

서버 함수 2개 — 저장 · 검색+답변

▶ 함정 ②: 임베딩은 배열이 아니라 "[...]" 문자열로 저장해야 vector 캐스팅됨

```
// api/add.js — 저장(색인)
const e = await openai.embeddings.create({ model:"text-embedding-3-small", input:text });
await supa.from("memos").insert({ text, embedding: JSON.stringify(e.data[0].embedding) });
// ↑ 배열 아니라 문자열!

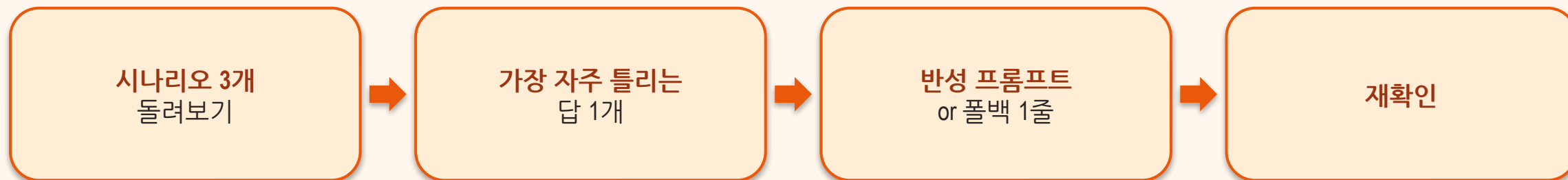
// api/ask.js — 검색 + 답변(RAG)
const { data: hits } = await supa.rpc("match_memos", { query_embedding: qv, match_count: 3 });
// hits(근거 메모)만 컨텍스트로 넣어 LLM 호출 → 근거 [n] 붙여 답변
```

SECTION 04

STEP 3 · ~21:00 — 반성 · 시나리오 · 제출

반성/가드 1개 — 빛나간 답 하나만 잡기

▶ 무한 반성 금지 — 반복 1회면 충분(13차시)



🔗 1회만 — 비용·지연도 함께 체크

예: "검색 결과 0건이면 '모르겠다'로 답" · "숫자 답은 두 번 확인"

에이전트 구현 과정 정리

1

문제

어제 MVP는 ○○를 못 했다

2

에이전트

T? 트랙으로 붙였다 — 왜 T?를 골랐는가

3

Wow

배포 URL에서 ○○ 순간이 이렇게 도는 걸 지금 보여준다

오늘 제출물

- ☐ 에이전트 기능 커밋 해시 (Day 2 브랜치 / PR 링크)
- ☐ 재배포된 URL — 어제와 동일
- ☐ AGENTS.md 업데이트 diff

SECTION 05

마무리

클로징 — 오늘 밤의 한 사이클

- 어제 MVP → 에이전트가 일하는 서비스 → 같은 URL 재배포 → 에이전트 구조 한 줄